

Artificial Intelligence

AI2002

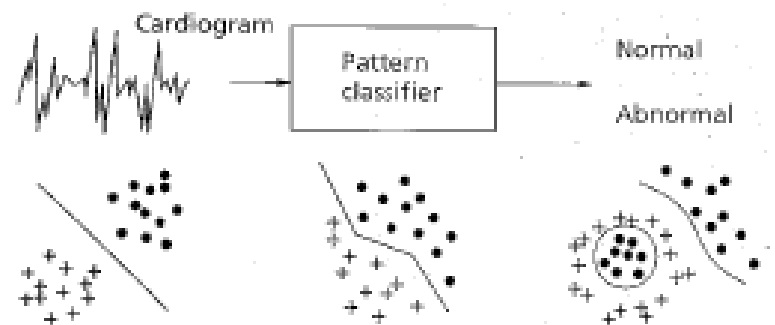
Rushda Muneer

Spring 2026

Types of Problems Solved by NN

Pattern Classification

- The task of pattern classification is to assign an input pattern (represented by a feature vector) to one of many pre-specified classes
- Applications include speech recognition (speech waveforms), character recognition (handwritten symbol), EEG waveform classification, blood cell classification



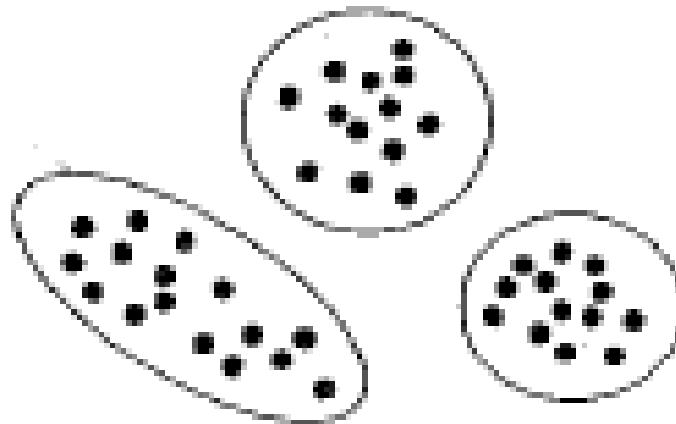
Pattern Completion

- To complete the missing parts of a given input vector
- The NN acts as an associative memory, whose contents can be recalled even by a partial input or distorted content.



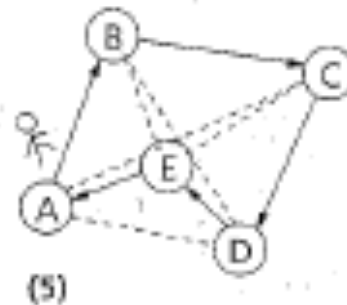
Clustering

- In clustering, the training data is unsupervised (class labels are not available)
- The similarity between patterns is explored and similar patterns are grouped together, thus forming clusters



Optimization

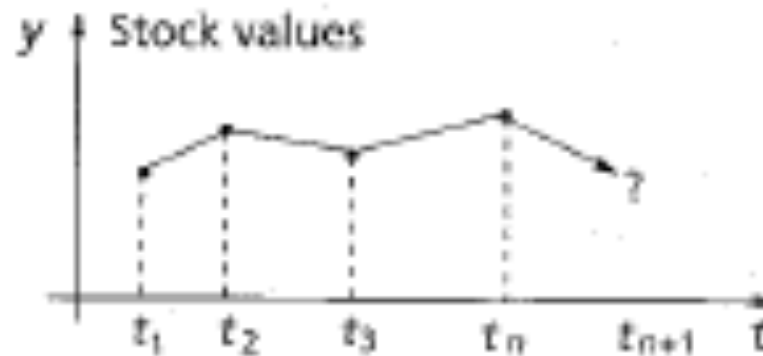
- To find the **optimal values** of **parameters**. A wide variety of problems can be posed as optimization problems
- The goal of an optimization algorithm is to find a solution satisfying a set of constraints such that an **objective function is maximized or minimized** (example TSP)
- Some NN architectures & their training algorithms can be used to solve optimization problems



(5) optimization (a TSP problem example);

Prediction / Forecasting

- Given a set of n samples in a time sequence, the task is to predict the sample $n + 1$
- Stock market prediction and weather forecasting are typical applications of prediction techniques



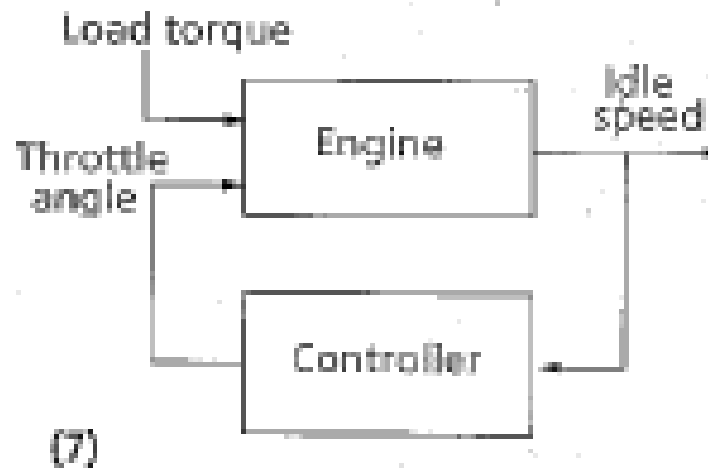
Function Approximation/Time Series Modeling

- To **learn the functional relationships** between input and desired output vectors
- Suppose a set of labeled training patterns (input-output pairs) have been generated from an unknown function (may be subject to noise). The task of FA is to find an estimate of the unknown function. Various engineering and scientific modeling problems require FA



Control

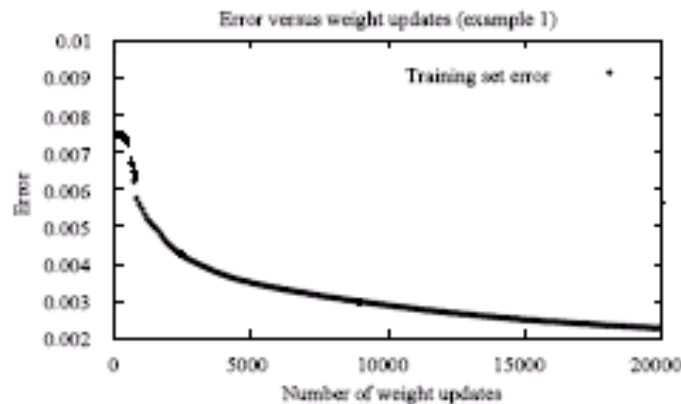
- An **appropriate action is suggested**, given an input vector
- Consider a dynamic system defined by a tuple $\{u(t), y(t)\}$, where $u(t)$ is the input from the controller and $y(t)$ is the resulting output of the system at time t . The goal is to generate a control input $u(t)$ such that the system follows a desired trajectory.



Issues with ANNs

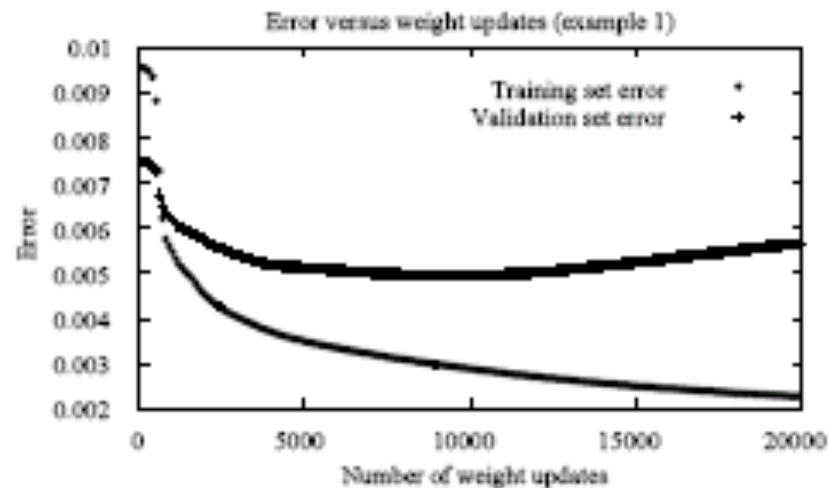
Generalization & Overfitting

- **Overfitting:** When to terminate the training of a network?
- The obvious choice is to stop, when the error falls below a certain threshold
- or
- when it stops reducing significantly (reaches a plateau)

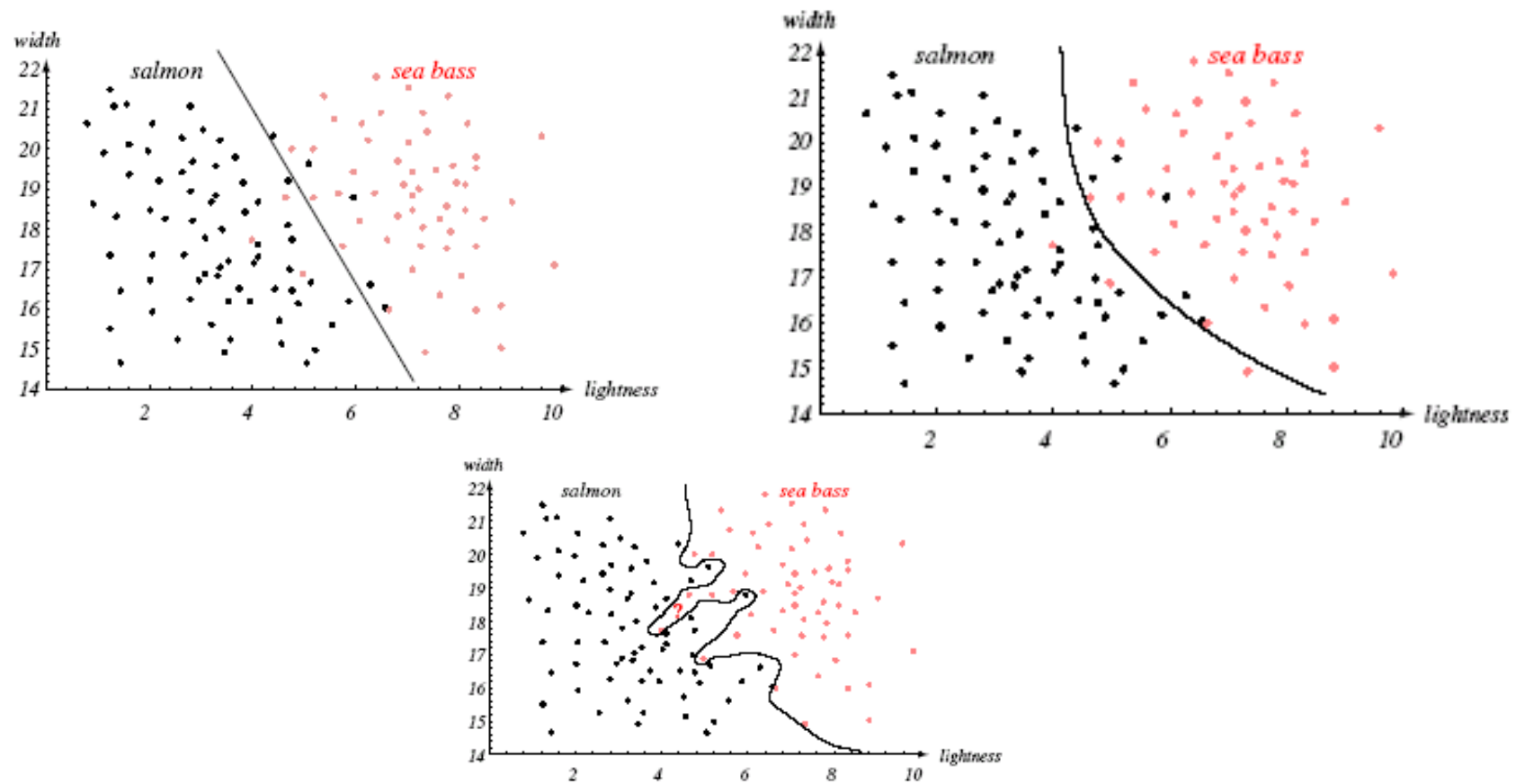


Generalization & Overfitting

- This is a poor strategy because **backpropagation tends to over-fit** the training examples in its **effort to reduce the error**
- The NN memorizes the training patterns & this results in a **loss of the ability to generalize**



Generalization & Overfitting

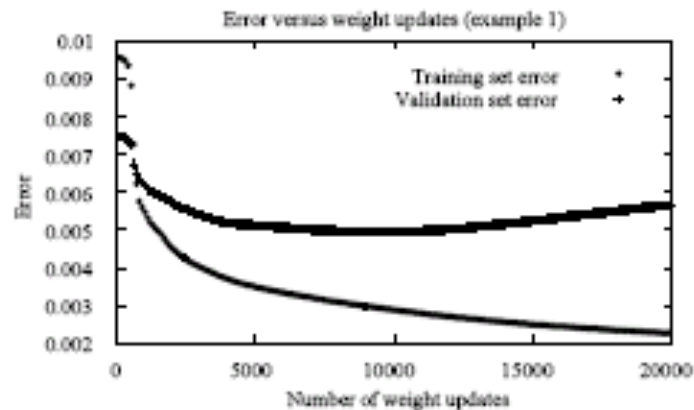


Generalization & Overfitting

- How to overcome the overfitting problem?
- Remedies for overfitting include **optimizing the network architecture** and using enough training patterns
- Another approach is to detect the point of overfitting by a set called **validation set**
- For this purpose, the original data set is divided into three disjoint sets: **training set, generalization set, and validation set**

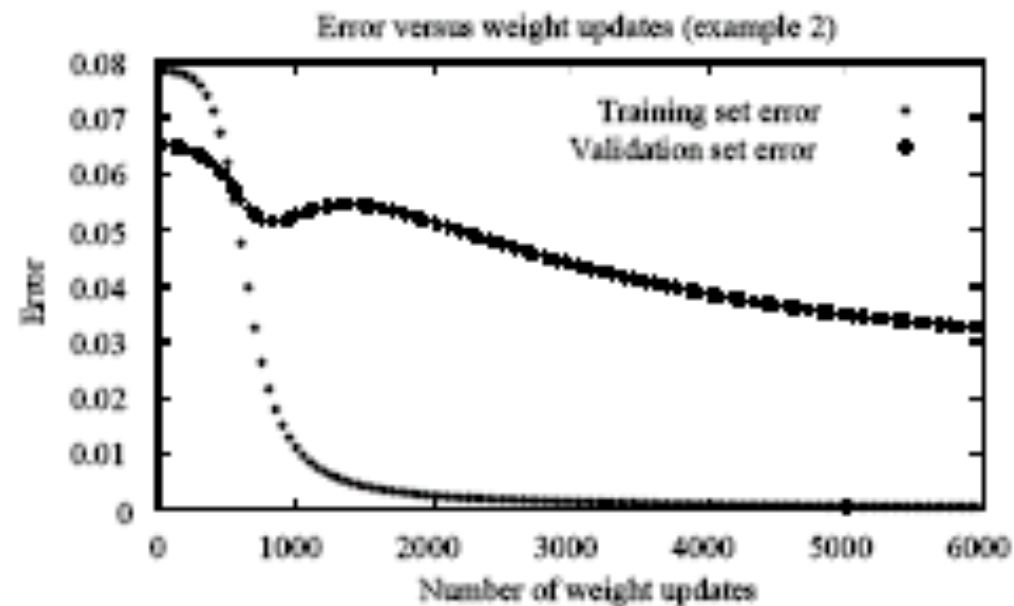
Generalization & Overfitting

- **Train on the training set but monitor the performance on the validation set**
- Save the weights after every few iterations
- The weights corresponding to the **lowest error on the validation set should be used as the final weights**



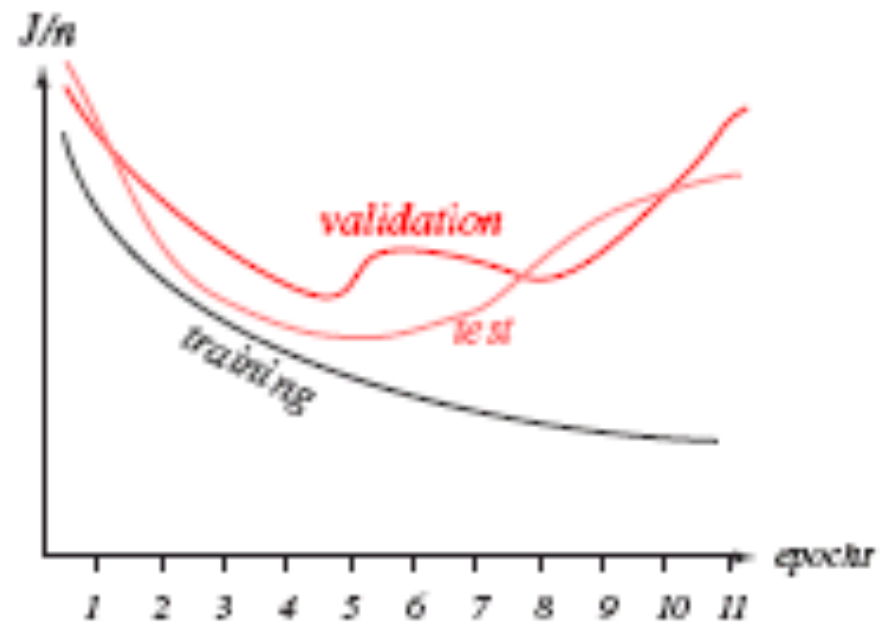
Generalization & Overfitting

- Attention: Validation error can sometimes increase and then reduce again



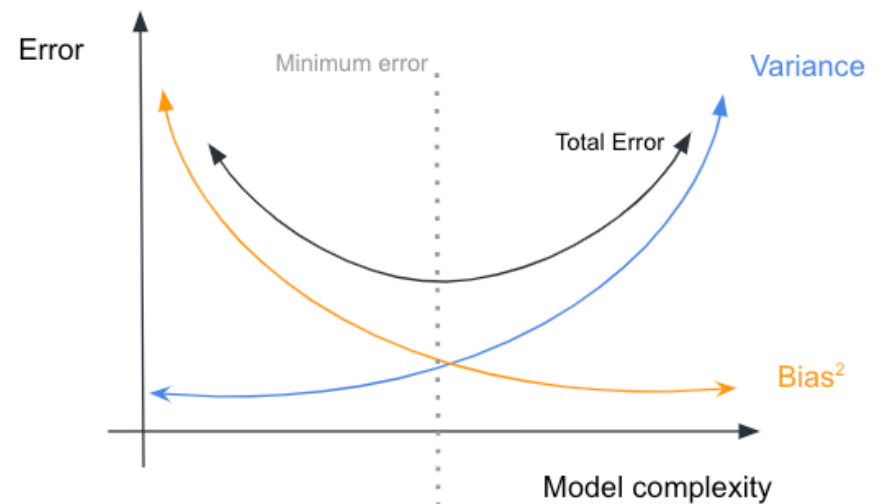
Generalization & Overfitting

- An example: Training set error, validation set error and test set error



Bias Variance Trade off

- This is a fundamental concept explaining model performance.
- **Bias:** Error due to overly simple assumptions.
- **Variance:** Error due to sensitivity to training data fluctuations.
- **Tradeoff:**
 - High Bias $\rightarrow$ Underfitting
 - High Variance $\rightarrow$ Overfitting
- **Goal:** Find a balance where both are minimized.



Vanishing Gradient Problem

- Vanishing gradients occur **when gradients become extremely small** during backpropagation, causing early layers to **learn very slowly or stop learning**.
- Gradients become very small during backpropagation.
- Weights stop updating effectively, especially in deep networks.
- **Common in:** Sigmoid and tanh activation functions
- When activation functions like Sigmoid or Tanh are used, their derivatives are less than 1.
- Repeated multiplication through layers causes the gradient to vanish as it moves backwards, making the lower layers unable to learn.
- **Solutions:**
 - Use ReLU or its variants
 - Proper weight initialization
 - Batch normalization

Exploding Gradient Problem

- Exploding gradients occur when **gradients grow too large** during backpropagation, leading to **unstable weight updates and divergence in loss**.
- When derivatives or weights are greater than 1, their repeated multiplication across layers leads to exponential growth.
- Gradients become extremely large.
- Causes unstable training and large weight updates.
- **Effects:** Model fails to converge
- **Solutions:**
 - Gradient clipping (**stay within a safe range**)
 - Better initialization
 - Use normalization techniques

Data Preparation

- One of the most important phases in using a NN is to collect and transform data into a form acceptable to the NN
- The **first step is to decide on the inputs and the outputs**. Obviously irrelevant inputs should be excluded
- The **second step is to process the data** in order to
 - **remove the outliers,**
 - **handle missing data,**
 - **transform non-numeric data to numeric,** and
 - **to scale the data into the active range of the activation function** used

Data Preparation: Missing Values

- For missing values in the input, the following options exists:
- **Remove the entire pattern**
- **Replace each missing value with the average value** (continuous value case) or the **most frequently occurring value** (discrete value case)
- For each input parameter that has a missing value, **add an additional input unit to indicate patterns for which that parameter is missing**. It can then be determined after training (by analyzing the weights on the links going out from that unit) whether the missing values had a significant influence on the network

Data Preparation: Coding of Nominal Input Values

- **All input values to a NN must be numeric. Nominal values therefore need to be transformed to numerical values**
- **A nominal input parameter that has n different values is coded as n different binary input parameters, where the input parameter that corresponds to a nominal value has the value 1, and the rest of these parameters have the value zero**

Example:	Apple	Input x1 x2 x3
	Red	1 0 0
	Green	0 1 0
	Yellow	0 0 1

Data Preparation: Coding of Nominal Input Values

- An alternative is to use just **one input parameter** and to map each nominal value into an **equivalent numerical value**. This is, however, **not a good idea, since the NN will interpret the input parameter as having continuous values, thereby losing the discrete characteristic of the original data**

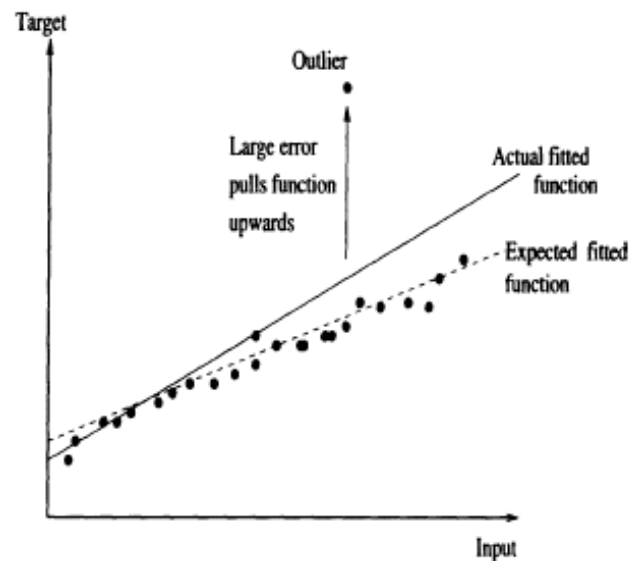
Example:	Apple	Input x
	Red	1
	Green	2
	Yellow	3

Data Preparation: Input Output Scaling

- It is useful to scale **the input values to the active range** (e.g. -1, 1) and domain (e.g. $-3^{1/2}$, $3^{1/2}$ for hyperbolic tangent) of the activation function used
- Output values have to be scaled to **the active range of the activation function**

Data Preparation: Outliers

- **Outliers** have severe effects on accuracy, because they result in **large errors, and consequently large weight updates**.
- The learned function is pulled towards the outliers in an attempt to reduce the training error

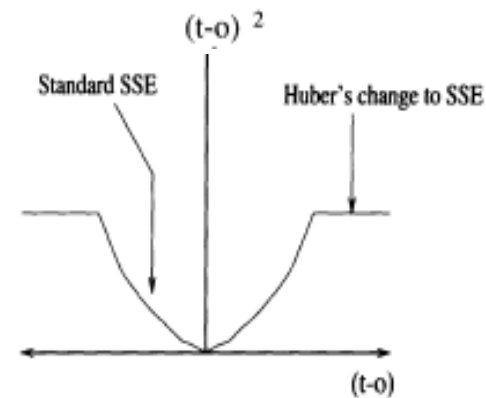


Data Preparation: Outliers

- Solution:
- **Remove outliers before the start of the training**, using statistical techniques.
- This method has the risk of eliminating important information about the data (either the outlier may be a genuine pattern, or only one parameter in the outlier may be erroneous and the other parameter values may be correct and precious)

Data Preparation: Outliers

- Solution:
- Use a **robust error function** that is **not influenced by outliers**
- An example is the **Huber function**, shown in the figure
- **Patterns for which the error is larger than $|\alpha|$ have a constant value, and have zero influence when weights are updated (the derivative of a constant is zero)**



Data Preparation: Noise Injection

- For problems with a **limited number of training patterns, controlled injection of noise helps to generate new training patterns**
- It has been observed that addition of noise results in reduced training time and increased accuracy

Data Preparation: Training Set Presentation Order

- It is usually helpful to control the **order in which the training patterns are presented for learning**
- If there are three classes present, **it is desirable to have pattern of one class, followed by another, followed by third**
- **If all the patterns of a class are grouped together, the network will learn very slowly or not at all**
- Similarly, depending upon the problem, other manipulations may be done: e.g. **reduction of training patterns bunching together in the space**

Data Preparation: Training Set Size

- Rule of thumb has been proposed
- $P = W/e$ provided training accuracy is $1 - 0.5e$
- Where P = number of training patterns required
- W = number of weights to be trained
- $1 - e$ = classification accuracy expected
- Example: If $e = 0.1$, $W = 80$ then $P = 80/(0.1) = 800$ patterns are required if we want to assure 90% testing accuracy (the net has to be trained to classify 95% of training patterns correctly)

Activation Functions

1. Sigmoid Function

- Output range: (0, 1) $\sigma(x) = \frac{1}{1+e^{-x}}$

- **Characteristics:**

- Smooth and differentiable
- Used in binary classification

- **Problems:**

- Vanishing gradient
- Output not zero-centered

Activation Functions

2. Step Function

- Output: 0 or 1

$$f(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

- **Characteristics:**

- Simple decision boundary
- Used in early perceptrons

- **Problems:**

- Not differentiable → cannot use backpropagation

Activation Functions

3. ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$

- **Characteristics:**
 - Very popular in deep learning
 - Efficient and fast
- **Advantages:**
 - Reduces vanishing gradient problem
- **Problems:**
 - “Dying ReLU” (neurons stuck at 0)

Activation Functions

4. Tanh Function

- Output range: $(-1, 1)$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- **Characteristics:**
 - Zero-centered output
 - Stronger gradients than sigmoid
- **Problem:**
 - Still suffers from vanishing gradient

Activation Functions

5. Softmax Function

- Used for **multi-class classification**
- Converts outputs into probabilities

- **Properties:**
$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$
- Outputs sum to 1
- Each value represents probability of a class

One-Hot Encoding

- One-Hot Encoding is a way to represent categorical data as binary vectors.

Class	One-Hot Vector
Cat	[1, 0, 0]
Dog	[0, 1, 0]
Bird	[0, 0, 1]

- **Why Use One-Hot Encoding?**
- Neural networks cannot process categorical labels directly
- Prevents the model from assuming ordinal relationships